



Mobile web automation

Instalação, WebdriverIO, gestos, contexts, POM e testflow-appium

Android · iOS · TypeScript · Allure



O que é o Appium?

Framework open-source de automação mobile que usa o protocolo **WebDriver** — funciona com apps nativos, híbridos e **mobile web** (Chrome/Safari).

- **Appium 2** — arquitetura modular com drivers plugáveis
- Drivers: `uiautomator2` (Android) · `xcuitest` (iOS)
- Cliente: **WebdriverIO 9** + Mocha neste projeto
- TestFlow via browser mobile — mesma app dos suites Cypress/Playwright
- Screenshots on failure, Allure report, tags Mocha (`@smoke` , `@appium`)



Por que Appium + WebDriverIO?



Dispositivos reais

Emulador Android, Simulator iOS ou device físico — gestos touch nativos.



Mobile web

Chrome no Android, Safari no iOS — POM com `data-testid`.



Appium 2 local

`@wdio/appium-service` sobe o server automaticamente no `wdio run`.



Allure + CI

Relatório HTML no GitHub Pages; smoke Android no PR gate.



Pré-requisitos

Ambiente

 **Node.js** 20+

 **Java 17+** (Allure report)

 **Android:** SDK, emulator, Chrome

 **iOS:** macOS, Xcode, Simulator

 TestFlow na porta **5050**

```
$ node --version
v20.x.x

$ docker run -p 5050:5050 gaschool/testflow:latest
# npm run check:testflow
```



Instalação do ambiente por SO

Antes de clonar: **Node 20+**, **Git**, **Docker** (TestFlow) e stack mobile conforme plataforma alvo.



macOS — Android + iOS

Base

```
brew install node@20 git
brew link --overwrite node@20
brew install --cask docker android-studio

# Java 17 (Allure)
brew install openjdk@17
```

Appium drivers (no projeto)

```
npm install
npm run appium:drivers # uiautomator2 + xcuitest
npm run appium:doctor # valida drivers

# iOS - Xcode + Simulator
xcode-select --install
open -a Simulator
```



Windows — Android

```
winget install OpenJS.NodeJS.LTS
winget install Git.Git
winget install Docker.DockerDesktop

# Android Studio + SDK Manager
# ANDROID_HOME → %LOCALAPPDATA%\Android\Sdk
```

```
npm install
npm run appium:drivers

# Emulador: Pixel 7, API 34
# adb reverse tcp:5050 tcp:5050 # CI pattern
```

iOS exige macOS — use CI ou Mac para `npm run test:ios`.



Linux — Android (CI)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt-get install -y nodejs git openjdk-17-jdk  
  
npm install  
npm run appium:drivers
```

```
# GitHub Actions usa reactivecircus/android-emulator-runner  
# KVM + API 34 + pixel_7 profile  
# scripts/ci-android-smoke.sh
```




Instalação — testflow-appium

1 Clone e instale

```
git clone .../testflow-appium && cd testflow-appium && npm install
```

2 Drivers Appium

```
npm run appium:drivers
```

3 Configure env

```
cp .env.example .env
```

4 Suba TestFlow e rode

```
docker run -p 5050:5050 gaschoo1/testflow:latest
```

```
npm run test:api OU npm run test:smoke
```



URLs mobile — loopback

Emuladores não resolvem `localhost` como o desktop.

Plataforma	Base URL padrão
Android emulator	<code>http://10.0.2.2:5050</code>
iOS simulator	<code>http://127.0.0.1:5050</code>
CI (adb reverse)	<code>http://127.0.0.1:5050</code>
API only / desktop	<code>http://localhost:5050</code>

Override: `BASE_URL`, `ANDROID_BASE_URL`, `IOS_BASE_URL`, `API_BASE_URL`



Estrutura do projeto

```
testflow-appium/  
├─ wdio.conf.ts      # Android  
├─ wdio.ios.conf.ts  # iOS Safari  
├─ wdio.api.conf.ts   # REST-only  
├─ wdio.shared.conf.ts # Appium service + Allure  
├─ pages/             # Page Object Model  
├─ support/  
│   ├─ auth.ts        # login API/UI + sessionStorage  
│   ├─ selectors.ts    # data-testid helpers  
│   └─ navigation.ts   # openAppPath  
├─ helpers/           # gestures, waits, visual  
├─ tests/             # specs por feature  
└─ scripts/           # CI + Allure
```

- `pages/` — ações e assertions por tela
- `support/auth.ts` — seed de sessão via API
- `tests/*.spec.ts` — Mocha + expect-webdriverio
- Tags: `@smoke` · `@regression` · `@appium` · `@api`



Configuração — wdio.conf.ts

```
const androidCaps = {
  platformName: 'Android',
  browserName: 'Chrome',
  'appium:automationName': 'UiAutomator2',
  'appium:deviceName': process.env.ANDROID_DEVICE ?? 'Android Emulator',
  'appium:chromedriverAutodownload': true,
  'appium:pageLoadStrategy': 'eager',
  'wdio:enforceWebDriverClassic': true,
}

export const config = {
  ...sharedConfig,
  capabilities: [androidCaps],
}
```

`sharedConfig`: Appium service, Mocha, Allure reporter, screenshot on failure.



Seu primeiro teste

```
// tests/auth/login.spec.ts
import { LoginPage } from '../../pages/LoginPage'
import { DEMO_EMAIL, DEMO_PASSWORD } from '../../support/config'

describe('Authentication @regression', () => {
  const login = () => new LoginPage()

  beforeEach(async () => {
    await login().visit()
  })

  it('logs in via UI and redirects to dashboard', async () => {
    await login().loginWith(DEMO_EMAIL, DEMO_PASSWORD)
    await login().shouldRedirectToDashboard()
  })
})
```



Navegação & browser

API	Uso no TestFlow
<code>browser.url(url)</code>	Abrir login, dashboard via <code>openAppPath</code>
<code>browser.getUrl()</code>	Assert redirect pós-login
<code>browser.getTitle()</code>	Smoke — título da página
<code>browser.execute(fn)</code>	sessionStorage, DOM queries, userAgent
<code>browser.waitUntil(fn)</code>	document.readyState, poll custom
<code>browser.takeScreenshot()</code>	Allure attachment on failure
<code>browser.saveScreenshot(path)</code>	Visual baselines, device spec
<code>browser.capabilities</code>	platformName, device metadata



Elementos — find & interact

API	Exemplo TestFlow
<code>browser.\$(selector)</code>	<code>[data-testid="login-email"]</code>
<code>browser.\$\$ (prefix)</code>	Múltiplos elementos por prefixo
<code>.click()</code>	Submit, sidebar, nav links
<code>.setValue(text)</code>	Email, senha, busca
<code>.clearValue()</code>	Limpar campo antes de fill
<code>.getText()</code>	KPIs, erros, labels
<code>.getAttribute(name)</code>	Classe active na nav
<code>.waitForDisplayed()</code>	Explicit wait por elemento
<code>.waitForEnabled()</code>	Campo habilitado antes de interagir
<code>.isDisplayed()</code> / <code>.isExisting()</code>	Condicionalis em locators spec



Assertions — expect-webdriverio

Assertion	Exemplo
<code>toBeDisplayed()</code>	Page root, KPI card, modal
<code>toBeEnabled()</code>	Campo password após wait
<code>toHaveUrl(...)</code>	Redirect pós-login / logout
<code>toHaveText(...)</code>	Mensagem de erro no login
<code>expect(browser).toHaveUrl(...)</code>	URL contém <code>/web/team.html</code>
<code>assert.strictEqual</code>	Contratos API (Mocha + Axios)



Helpers — support/selectors.ts

Seletores data-testid	
<code>byTestId(id)</code>	CSS <code>[data-testid="..."]</code>
<code>\$(testId)</code>	Atalho para <code>browser.\$</code>
<code>waitForTestId(id)</code>	<code>waitForExist</code> + <code>waitForDisplayed</code>
<code>clickTestId(id)</code>	Click com wait implícito
<code>fillTestId(id, value)</code>	<code>click</code> → <code>clear</code> → <code>setValue</code>
<code>getTextTestId(id)</code>	Texto de elemento estável
support/helpers/waits.ts	
<code>waitForUrl(regex)</code>	Poll até URL match
<code>waitForText(testId, text)</code>	Poll até texto aparecer
<code>pollUntil(fn, pred)</code>	Retry genérico com intervalo



Gestos touch — performActions

```
// support/helpers/gestures.ts
export async function swipeUp({ percent = 0.4, speed = 600 } = {}) {
  const { width, height } = await browser.getWindowSize()
  await browser.performActions([
    { type: 'pointer', id: 'finger1',
      parameters: { pointerType: 'touch' },
      actions: [
        { type: 'pointerMove', duration: 0, x: width/2, y: height*0.75 },
        { type: 'pointerDown', button: 0 },
        { type: 'pointerMove', duration: speed, x: width/2, y: height*(1-percent) },
        { type: 'pointerUp', button: 0 },
      ],
    },
  ])
  await browser.releaseActions()
}
```

swipeUp

swipeDown

longPress

scrollIntoView



Device capabilities

Função / API	Spec
<code>hideKeyboardIfShown()</code>	<code>device.spec.ts</code> — teclado software
<code>setOrientation('PORTRAIT' 'LANDSCAPE')</code>	Rotação portrait/landscape
<code>browser.isKeyboardShown()</code>	Detectar teclado aberto
<code>browser.hideKeyboard()</code>	Fechar teclado Android
<code>browser.setOrientation(...)</code>	Wrapper Appium
<code>browser.saveScreenshot(path)</code>	Screenshot sob demanda
<code>mobile: backgroundApp</code>	Simula background (native; web ignora)



Context switching

```
// support/helpers/gestures.ts
export async function listContexts() {
  return (await browser.getContexts()).map(String)
}

export async function switchToWebContext() {
  const web = (await listContexts())
    .find((ctx) => ctx.includes('WEBVIEW')) || ctx === 'WEBVIEW'
  if (web) await browser.switchContext(web)
}

// tests/contexts/webview.spec.ts
const testIds = await browser.execute(() =>
  Array.from(document.querySelectorAll('[data-testid]'))
    .slice(0, 5).map(el => el.getAttribute('data-testid'))
)
```



Estratégias de locator

Estratégia	Exemplo
CSS / data-testid	<code>browser.\$(byTestId('login-email'))</code>
XPath	<code>//input[@data-testid="login-email"]</code>
Atributo composto	<code>button[data-testid="login-submit"]</code>
Explicit wait	<code>.waitForDisplayed({ timeout: 10000 })</code>

Preferência: `data-testid` — mesma convenção Cypress/Playwright.



Auth helpers

```
// support/auth.ts
export async function loginViaApi() {
  const session = await fetchAuthToken() // Axios POST /api/auth/login
  await seedSessionOnOrigin() => injectAuthSession(session)
  await openAppPath('/web/dashboard.html')
  await waitForTestId('page-dashboard')
}

export async function visitWithToken(path, token) {
  await seedSessionOnOrigin() => injectAuthSession({ token, ... })
  await openAppPath(path)
}
```

`injectAuthSession` grava `sandbox-auth` e `sandbox-token` no sessionStorage.



API testing — sem device

```
// tests/api/auth.api.spec.ts - Mocha + Axios
describe('API - health & auth @api @smoke', () => {
  it('GET /health returns 200', async () => {
    const res = await apiClient().get('/health')
    assert.strictEqual(res.status, HTTP.OK)
  })

  it('POST /api/auth/login returns token', async () => {
    const res = await apiClient().post('/api/auth/login', {
      email: DEMO_EMAIL, password: DEMO_PASSWORD,
    })
    validateSchema(res.data, { token: 'string', user: 'object' })
  })
})
```

```
npm run test:api # não precisa de emulador
```



Visual baselines

```
// support/helpers/visual.ts
export async function assertScreenshotMatches(name, maxMismatchPercent = 5) {
  const baselinePath = `test-results/visual-baselines/${name}.png`
  if (!fs.existsSync(baselinePath)) {
    await saveBaseline(name) // cria baseline na 1ª execução
    return
  }
  await browser.saveScreenshot(`${name}-current.png`)
  // compara tamanho de arquivo como proxy de diff
}
```

npm run test:visual

@visual



Page Object Model (POM)

Spec (.spec.ts)

- describe / it (Mocha)
- Given / When / Then
- Sem seletores espalhados



Page Object

- `testId('login-email')`
- Actions (`loginWith`)
- Assertions (`shouldRedirect...`)



Page Objects do TestFlow

LoginPage.ts auth, validação, API toggle

DashboardPage.ts KPIs, activity, new run

TeamPage.ts tabela, invite, inline edit

SettingsPage.ts forms, toggles, save

WizardPage.ts multi-step, review

ShellPage.ts nav, theme, mobile shell



Cobertura da suíte

Suite	Spec	Tag
Smoke	navigation.spec.ts	@smoke
Auth	login.spec.ts	@regression
Dashboard / Team / Settings	*.spec.ts	@regression
Gestures	gestures.spec.ts	@appium
Device	device.spec.ts	@appium
Contexts	webview.spec.ts	@appium
Locators	strategies.spec.ts	@appium
Visual	visual.spec.ts	@visual
API	auth.api.spec.ts	@api



Todos os comandos npm

Comando	Descrição
Setup	
<code>npm run appium:drivers</code>	Instala uiautomator2 + xcuitest
<code>npm run appium:doctor</code>	Valida drivers Appium
<code>npm run check:testflow</code>	Health check porta 5050
Execução	
<code>npm test</code>	Suite Android completa
<code>npm run test:android</code>	Android mobile web
<code>npm run test:ios</code>	iOS Safari (macOS)
<code>npm run test:api</code>	REST — sem emulador
<code>npm run test:smoke</code>	Só smoke specs
<code>npm run test:gestures</code>	Gestos touch
<code>npm run test:device</code>	Teclado, orientação



Todos os comandos npm

Comando	Descrição
Execução	
<code>npm run test:contexts</code>	Context switching
<code>npm run test:locators</code>	Estratégias de locator
<code>npm run test:grep:smoke</code>	Filtra tag <code>@smoke</code>
<code>npm run test:grep:appium</code>	Filtra tag <code>@appium</code>
Relatório	
<code>npm run allure:generate</code>	Gera HTML Allure
<code>npm run allure:open</code>	Abre relatório
<code>npm run report</code>	generate + open
Slides	
<code>npm run slides</code>	Servidor local :3336
<code>npm run slides:pdf</code>	Exporta <code>appium-intro-slides.pdf</code>



Allure Report

```
npm run test:smoke
npm run allure:generate # Java 17+
npm run allure:open

# atalho
npm run report
```

- Screenshots anexados em falha (`afterTest`)
- URL atual no attachment
- Env vars no report (platform, CI, URLs)
- GitHub Pages: push em `main` → `pages.yml`



CI — GitHub Actions

Job api

REST smoke contra TestFlow Docker — gate rápido

android-smoke

Emulator API 34, KVM, `ci-android-smoke.sh`

publish / deploy

Landing, guides, slides + Allure (or placeholder) na `main`

Artefatos

Screenshots + allure-results em falha



Boas práticas

- **Page Objects** — locators `data-testid` centralizados
- `loginViaApi` no `beforeEach` — specs focam na feature
- `waitForTestId` — explicit wait antes de interagir
- URLs por plataforma em `support/config.ts` — não hardcode localhost
- Gestos via `performActions` — W3C Actions API
- `api` e `android-smoke` em paralelo no CI
- Tags Mocha — filtrar `@smoke`, `@appium` no CI
- IDs rastreáveis `[TC-xxxx]` em `testCases.ts`
- Um cenário por `it()` — falhas legíveis no Allure



Próximos passos

Você já sabe instalar Appium, configurar WebdriverIO e rodar mobile web no TestFlow.



Slides (PDF)

appium-intro-slides.pdf



Guia completo (PT)

Instalação e referência



Documentação

appium.io



WebdriverIO

webdriver.io



TestFlow

App sob teste

← → slides | ↓ ↑ sub-slides | F tela cheia | ESC overview